

Basic Model

BasicModel

Basic Model of Cognition

Logo

The basic model of cognition relies on conceptual hyperplanes and perceptual prototypes to synthesize and analyze the input space. It uses a high-dimensional embedding to characterize mental space and integrates symbolic computation. Its three major operations are intersection (which forms percepts from concepts), union (a bidirectional mapping between concepts and percepts), and equality (where symbols are elements that map across perceptual and conceptual domains).

Overview

BasicModel is a parameterized neural architecture with three independent levers:

- **Ergodic** — adaptive bias-variance control via a gradient energy sensor (not gradient descent)
- **Certainty** — certainty-weighted cross-entropy loss
- **Quantized** — vector quantization of the conceptual space

Model configurations are specified in XML and can be compared side-by-side. See doc/Architecture.md for the full mathematical treatment.

Files

File	Description
bin/BasicModel.py	Main entry point: DerivedModel, training loop, comparison plots, HTML report
bin/Model.py	Layer library: SigmaLayer, ErgodicLayer, LinearLayer, spaces, and utilities
bin/SigmaPi.py	Standalone demo of the SigmaPi network solving XOR
data/	XML model configurations and static embeddings
doc/Architecture.md	Algorithm details: Sigma/Pi layers, ergodic exploration, gradient energy sensor
test/	Unit tests

Quick Start

```
# Set up virtual environment
make venv

# Run a single model
make simple      # data/simple.xml
make ergodic     # data/ergodic.xml

# Compare two models side-by-side
make compare     # defaults: data/simple.xml vs data/ergodic-only.xml
```

```

make compare XML1=data/simple.xml XML2=data/ergodic.xml

# Run tests
make test

# Generate PDF documentation
make doc_pdf

```

XML Configuration

Models are configured via XML files in `data/`:

```

<model>
  <architecture>
    <nConcepts>20</nConcepts>
    <ergodic>true</ergodic>
    <certainty>true</certainty>
    <quantized>>false</quantized>
    <normed>>false</normed>
    <reverse>>false</reverse>
    <invert>>false</invert>
  </architecture>
  <training>
    <dataset>mnist</dataset>
    <numTrials>1</numTrials>
    <numEpochs>20</numEpochs>
    <batchSize>10</batchSize>
  </training>
</model>

```

Output

Each run produces an HTML report (timestamped in `output/`) containing:

- **Error per Epoch** — training and test loss curves
- **Accuracy per Digit** — per-class accuracy breakdown

In compare mode, additional overlay plots show combined loss and accuracy across models with color-coded legends.

Architecture

Sigma and Pi Layers

Given a weight matrix ($W \in \mathbb{R}^{m \times n}$) and input vector ($x \in \mathbb{R}^n$), the output vector ($y \in \mathbb{R}^m$) is computed as:

For the Sigma layer:

$$y_j = Wx + b$$

$$y_j = b_j + \sum_{i=1}^n W_{ji}x_i$$

For the Pi layer:

$$y_j = b_j \cdot \prod_{i=1}^n (1 + W_{ji}x_i)$$

This is not invertible, but if we are allowed twice as many outputs as inputs, we may: do inversion as:

1. Define:

$$s_j := \sum_i \tanh^{-1}(w_{ji}x_i) = Wx$$

2. In forward pass, use:

$$y_j = b_j \cdot \prod_i (1 - \tanh(w_{ji}x_i)), \quad z_j = b_j \cdot \prod_i (1 + \tanh(w_{ji}x_i))$$

3. In reverse pass, compute:

$$\gamma_j = \frac{1}{2} \log \left(\frac{z_j}{y_j} \right) = \sum_i \tanh^{-1}(\tanh(w_{ji}x_i)) = \sum_i w_{ji}x_i \Rightarrow \gamma = Wx$$

4. Finally:

$$x = W^{-1}\gamma$$

Dimensionality Constraints

- The output dimensionality of the input layer must be equal to the output dimensionality of the perceptual layer, since the conceptual layer operates on both.
 - The input dimensionality of the symbolic layer must be equal to the input dimensionality of the perceptual layer, since they both operate on the output of the conceptual layer.
 - The input dimensionality of the output layer must be equal to the sum of the output dimensionalities of the symbolic layers.
-

Pi Layer Derivation

$$e^{y_j} = e^{b_j} + \sum_{i=1}^n e^{1+W_{ji}x_i}$$

$$\log(e^{y_j}) = \log(e^{b_j} + \sum_{i=1}^n e^{1+W_{ji}x_i})$$

$$\log(e^{y_j}) = \log(e^{b_j}) \cdot \log \left(\sum_{i=1}^n e^{1+W_{ji}x_i} \right)$$

$$y_j = b_j \cdot \prod_{i=1}^n \log(e^{1+W_{ji}x_i})$$

$$y_j = b_j \cdot \prod_{i=1}^n (1 + W_{ji}x_i)$$

Now, is there a way to write $e^{1+W_{ji}x_i}$ as a matrix product, $W_{ji}x_i$?

$$e^{1+W_{ji}x_i} = e \cdot e^{W_{ji}x_i}$$

If we write W and x in the log domain first, we have:

$$e \cdot e^{\log(W_{ji})\log(x_i)} = e \cdot (W_{ji} + x_i)$$

DerivedModel: Ergodic Exploration via Adaptive Bias-Variance Control

BasicModel implements a novel approach to neural network optimization that decouples **what** the network learns (weights W) from **how much it explores** (a scalar α governing the bias-variance tradeoff). Two fundamentally different algorithms operate in tandem:

Component	Algorithm	What it does
Weights W	Standard Adam	Gradient descent on the loss landscape
Tradeoff α	Gradient Energy Sensor	Measures loss-surface curvature to set exploration level

The key insight: α **is not trained by gradient descent**. It is a *sensor* that reads the gradient energy flowing through the temperature parameter and converts it into an exploration policy.

1. The Ergodic Weights

In this model, weights are not treated as fixed constants. Each layer instead uses an effective weight matrix sampled from a locally specified ergodic distribution, with the learned tensor and the stochastic tensor defining the current mixture:

$$W_{\text{eff}} = \alpha \cdot W + (1 - \alpha) \cdot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

So the layer never acts on a bare, timeless weight matrix; it acts on the current sample W_{eff} , while W stores the learned structure that increasingly shapes that local distribution as α rises. This perspective is adjacent to the broader literature on stochastic sampling in learning, including Monte Carlo methods, stochastic approximation, and noise-injected views of neural network optimization.

We define two derived quantities from the single scalar α :

$$\text{bias} = \alpha, \quad \text{temp} = 1 - \alpha$$

This is a **direct encoding of the bias-variance tradeoff**:

- $\alpha = 0$: Pure exploration. $W_{\text{eff}} = \varepsilon$. The network is random noise.
- $\alpha = 1$: Pure exploitation. $W_{\text{eff}} = W$. The network uses only learned weights.
- $0 < \alpha < 1$: Interpolation between learned structure and stochastic exploration.

Training begins at $\alpha = 0$ (full exploration) and the system autonomously transitions toward exploitation as the loss landscape flattens.

2. Why Standard Adam Fails on α

The temperature parameter $T = 1 - \alpha$ receives gradients via:

$$\frac{\partial \mathcal{L}}{\partial T} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \cdot \varepsilon_{ij}$$

Because ε is **re-sampled every forward pass**, this gradient is **zero-mean**:

$$\mathbb{E}\left[\frac{\partial \mathcal{L}}{\partial T}\right] = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \cdot \underbrace{\mathbb{E}[\varepsilon_{ij}]}_{=0} = 0$$

Standard Adam maintains a first moment (running mean) and a second moment (running variance), then computes the update as $m/\sqrt{v}$. For a zero-mean gradient:

$$m_t \approx 0 \implies \frac{m_t}{\sqrt{v_t}} \approx \frac{0}{\sqrt{v_t}} = 0$$

Adam produces no update. The first moment kills the signal. This is correct behavior for Adam — it correctly identifies that there is no consistent gradient direction — but it means Adam cannot be used to tune α .

3. The Gradient Energy Sensor

3.1 Modified Second-Moment Estimator We discard the first moment entirely and use only the second moment, but as **output** rather than normalizer:

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot g_t^2$$

where $g_t = \partial \mathcal{L} / \partial T$ is the temperature gradient at step t , and $\beta = 0.999$ is the EMA decay rate.

3.2 Bias Correction Identical to Adam’s bias correction. Since $v_0 = 0$, early estimates are biased toward zero:

$$\hat{v}_t = \frac{v_t}{1 - \beta^t}$$

3.3 What $\hat{v}_t$ Measures The second moment of a zero-mean random variable is its variance:

$$\mathbb{E}[g_t^2] = \text{Var}(g_t) = \sum_{i,j} \left(\frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \right)^2 = \left\| \frac{\partial \mathcal{L}}{\partial W_{\text{eff}}} \right\|_F^2$$

This is the **squared Frobenius norm** of the loss gradient with respect to the effective weights — a scalar measure of how much the loss surface cares about weight perturbations. We call this the **gradient energy**.

Gradient energy	Meaning	Desired behavior
High $\hat{v}_t$	Loss is sensitive to weight changes	Keep exploring (low α)
Low $\hat{v}_t$	Loss surface is flat	Exploit learned weights (high α)

3.4 The Alpha Update Rule

$$\alpha_t = \frac{1}{1 + \tau \cdot \sqrt{\hat{v}_t}}$$

where τ (`global_temp`) is a tunable sensitivity knob.

Properties:

- $\sqrt{\hat{v}_t} \rightarrow 0 \implies \alpha_t \rightarrow 1$ (exploit when gradients vanish)
- $\sqrt{\hat{v}_t} \rightarrow \infty \implies \alpha_t \rightarrow 0$ (explore when gradients are large)
- $\alpha_t \in (0, 1)$ always — the sigmoid-like shape prevents saturation
- τ controls the transition speed: large τ means more exploration at equivalent gradient energy

After computing α_t , the bias and temperature are derived:

$$\text{bias}_t = \alpha_t, \quad T_t = 1 - \alpha_t$$

3.5 Layer-Local Certainty The scalar α_t is a **global layer policy**, but individual output neurons do not need to converge at the same rate. Each ergodic layer therefore maintains a certainty vector

$$c_t \in [0, 1]^{n_{\text{out}}}$$

whose entries are tuned from two separate signals kept outside the core ergodic update: a forward-pass activation statistic and the historical gradient energy of the learned per-output parameters in that layer.

The forward signal is a bounded summary of activation magnitude:

$$f_{t,j} = \tanh(\mathbb{E}[|y_{t,j}|])$$

The gradient signal is:

$$v_{t,j}^{(c)} = \beta_c v_{t-1,j}^{(c)} + (1 - \beta_c) \cdot g_{t,j}^2$$

$$\hat{v}_{t,j}^{(c)} = \frac{v_{t,j}^{(c)}}{1 - \beta_c^t}, \quad g_{t,j}^{(c)} = \frac{1}{1 + \tau_c \sqrt{\hat{v}_{t,j}^{(c)}}}$$

The final local certainty is a blend of the forward and gradient views:

$$c_{t,j} = \lambda_f f_{t,j} + \lambda_g g_{t,j}^{(c)}, \quad \lambda_f + \lambda_g = 1$$

This yields neuron-wise bias/variance coefficients:

$$\text{bias}_{t,j}^{\text{local}} = c_{t,j} \cdot \text{bias}_t$$

$$T_{t,j}^{\text{local}} = T_t + (1 - c_{t,j}) \cdot \text{bias}_t$$

so that $\text{bias}_{t,j}^{\text{local}} + T_{t,j}^{\text{local}} = 1$ whenever dropout is inactive. High-certainty outputs stabilize early and lean on learned weights; low-certainty outputs keep more noise and continue exploring. This lets earlier or easier features settle before later or less reliable ones without changing the global ergodic algorithm.

4. Comparison: Adam vs. Gradient Energy Sensor

Property	Adam (for W)	Gradient Energy Sensor (for α)
Type	Optimizer (gradient descent)	Sensor (measurement)
First moment m_t	Yes — tracks gradient direction	No — direction is zero-mean, useless
Second moment v_t	Yes — normalizes step size	Yes — is the output
Role of $\sqrt{v_t}$	Denominator (adaptive learning rate)	Numerator (gradient energy estimate)
Update rule	$W \leftarrow W - \eta \cdot m / \sqrt{v}$	$\alpha \leftarrow 1 / (1 + \tau \sqrt{\hat{v}})$
Descent?	Yes — follows gradient downhill	No — maps energy to a policy
Zero-mean safe?	No — produces $0 / \sqrt{v} = 0$	Yes — by design

5. Derivation: Why Zero-Mean Implies Drop m_t

Let $g_t = \nabla_T \mathcal{L}$ be the temperature gradient. Since noise ε is i.i.d. each step:

$$g_t = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \cdot \varepsilon_{ij}^{(t)}$$

The first moment EMA is:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Taking expectations:

$$\mathbb{E}[m_t] = \beta_1 \mathbb{E}[m_{t-1}] + (1 - \beta_1) \underbrace{\mathbb{E}[g_t]}_{=0} = \beta_1 \mathbb{E}[m_{t-1}]$$

By induction: $\mathbb{E}[m_t] = \beta_1^t \mathbb{E}[m_0] = 0$. The first moment converges exponentially to zero. It carries **no information** about the loss landscape — only noise from finite sampling. Keeping it would add variance to the estimator without adding signal.

The second moment, however, converges to a meaningful quantity:

$$\mathbb{E}[v_t] \rightarrow \mathbb{E}[g_t^2] = \|\nabla_{W_{\text{eff}}} \mathcal{L}\|_F^2$$

This is exactly the gradient energy — the quantity we need.

6. Dropout via Bias

The **bias** and **temp** parameters are passed through the full layer stack to support **dropout regularization**:

$$\text{bias}_t^{(\text{drop})} = \begin{cases} 0 & \text{with probability } p \\ \alpha_t & \text{with probability } 1 - p \end{cases}$$

When bias is dropped to zero, the layer acts as pure noise regardless of α . This is analogous to standard dropout but operates on the bias-variance tradeoff rather than individual activations. The temperature $T = 1 - \alpha$ remains unchanged so that the noise contribution is consistent.

Seen this way, dropout is just a schedule on bias. Ordinary Bernoulli dropout is the binary case in which the bias is either kept at α_t or forced to 0 for that step, while annealed dropout corresponds to increasing the expected bias over training so the layer spends less time in pure-noise mode and more time exploiting learned structure. More generally, nonzero temperature bakes regularization directly into the model because every forward pass perturbs the effective weights, discouraging brittle co-adaptation and rewarding structure that survives stochastic variation.

Every layer signature accepts (`bias`, `temp`) to support this:

```
def forward(self, x, bias=1.0, temp=0.0):
    W_eff = bias * W + temp * noise
    ...
```

7. The `global_temp` Sensitivity Knob

The parameter τ (`global_temp`) scales how responsive α is to gradient energy:

$$\alpha = \frac{1}{1 + \tau \cdot \sqrt{\hat{v}}}$$

τ	Effect
$\tau = 0$	$\alpha = 1$ always — pure exploitation, no exploration
$\tau \ll 1$	Aggressive exploitation — quickly converges to $\alpha \approx 1$
$\tau = 1$	Balanced — α tracks gradient energy on a natural scale
$\tau \gg 1$	Conservative — maintains high exploration even with moderate gradients

In practice, τ can itself be scheduled (e.g., annealed from high to low) to implement a coarse exploration-to-exploitation curriculum.

8. Initialization and Bootstrap

Problem: If we initialized at $\alpha = 1$ (pure exploitation), the noise term vanishes and the temperature gradient $\partial\mathcal{L}/\partial T = 0$. The sensor would read zero gradient energy and keep $\alpha = 1$ forever — an **exploitation trap**.

Solution: Initialize at $\alpha = 0$ (pure exploration):

```
self.alpha      = nn.Parameter(torch.tensor(0.0), requires_grad=False)
self.bias       = nn.Parameter(torch.tensor(0.0), requires_grad=False)
self.temperature = nn.Parameter(torch.tensor(1.0), requires_grad=True)
```

At $\alpha = 0$:

- $W_{\text{eff}} = \varepsilon$ — pure noise carries gradients through the network
- The temperature gradient is nonzero: $g_t = \sum_{ij} (\partial\mathcal{L}/\partial\varepsilon_{ij}) \cdot \varepsilon_{ij} \neq 0$
- The sensor measures initial gradient energy and begins tuning α upward
- As W learns useful structure, gradient energy decreases and α rises naturally

In this architecture, zero-initializing W is not only safe but often cleaner than random initialization. In an ordinary network, zero initialization is bad because it fails to break symmetry, but here symmetry is already broken by the sampled noise ε in W_{eff} . Since training begins at $\alpha = 0$, the learned weights do not contribute to the forward pass anyway, so random weight initialization only injects arbitrary structure into a phase that is meant to be pure exploration. Starting from $W = 0$ makes that exploratory regime honest and lets useful structure enter only when the bias toward learned weights increases.

9. Layer Architecture

All layers follow the effective-weight pattern with `(bias, temp)` signatures:

LinearLayer

$$y = x \cdot (\text{bias} \cdot W + \text{temp} \cdot \varepsilon_W) + \text{bias} \cdot b + \text{temp} \cdot \varepsilon_b$$

ReversibleLinearLayer (SVD decomposition)

$$W = U \Sigma V^\top$$

Each factor applies the tradeoff independently:

$$y = V \cdot \Sigma \cdot U^\top \cdot x$$

where each rotation and scaling layer uses `(bias, temp)` internally.

ReversibleRotationLayer Parameterized by Givens angles θ_k :

$$\theta_k^{\text{eff}} = \text{bias} \cdot \theta_k + \text{temp} \cdot \varepsilon_k$$

ReversibleDiagonalLayer Parameterized by log-singular-values λ_k :

$$\lambda_k^{\text{eff}} = \text{bias} \cdot \lambda_k + \text{temp} \cdot \varepsilon_k$$

10. Training Loop Integration

```
# 1. Standard Adam optimizer for W (excludes temperature)
optimizer = torch.optim.Adam(model.getParameters(), lr=lr)

# 2. Forward pass computes W_eff = bias*W + temp*noise
loss = model(x, y)

# 3. Backward pass: Adam gets dL/dW, temperature gets dL/dT
loss.backward()

# 4. Adam updates W
optimizer.step()

# 5. The sensor updates alpha, while local certainty blends
#     forward activity and gradient history
model.paramUpdate() # calls alpha_update() on each ErgodicLayer
```

Note that `getParameters()` **excludes** the temperature parameter:

```
def getParameters(self):
    params = [p for n, p in self.named_parameters() if n != "temperature"]
    return params
```

Temperature is excluded from Adam because Adam would produce zero updates on it (see Section 2). Instead, the gradient energy sensor reads `temperature.grad` directly.

11. Summary of the Full Algorithm

Initialize:

$$\alpha_0 = 0, \quad v_0 = 0, \quad t = 0, \quad \beta = 0.999$$

Each training step:

1. Sample $\varepsilon \sim \mathcal{N}(0, I)$
2. Compute $W_{\text{eff}} = \alpha_t W + (1 - \alpha_t)\varepsilon$
3. Forward pass: $\hat{y} = f(x; W_{\text{eff}})$
4. Compute loss $\mathcal{L}(\hat{y}, y)$
5. Backward pass: compute $\nabla_W \mathcal{L}$ and $g_t = \nabla_T \mathcal{L}$
6. **Adam** updates W : $W \leftarrow W - \eta \cdot \hat{m}_t / \sqrt{\hat{v}_t^{(W)}}$
7. **Sensor** updates α , and a separate certainty routine blends forward activity with gradient history:

$$v_t \leftarrow \beta \cdot v_{t-1} + (1 - \beta) \cdot g_t^2$$

$$\hat{v}_t = \frac{v_t}{1 - \beta^t}$$

$$\alpha_{t+1} = \frac{1}{1 + \tau \sqrt{\hat{v}_t}}$$

$$c_{t+1,j} = \frac{1}{1 + \tau_c \sqrt{\hat{v}_{t,j}^{(c)}}}$$

$$\text{bias}_{t+1,j}^{\text{local}} = c_{t+1,j} \cdot \text{bias}_{t+1}, \quad T_{t+1,j}^{\text{local}} = T_{t+1} + (1 - c_{t+1,j}) \cdot \text{bias}_{t+1}$$

8. Zero temperature gradient: $g_t \leftarrow 0$

Invertible Transformation Based on Hyperbolic Products

Forward Transformation

Given input vector $x \in \mathbb{R}^n$, parameters $W \in \mathbb{R}^{m \times n}$, and positive constants $b_j > 0$, define:

$$y_j = b_j \prod_{i=1}^n (1 - \tanh(w_{ji} x_i))$$

$$z_j = b_j \prod_{i=1}^n (1 + \tanh(w_{ji} x_i))$$

This mapping is invertible under mild assumptions on W .

Inverse Transformation

We can recover a latent variable γ_j as:

$$\frac{z_j}{y_j} = \prod_{i=1}^n \frac{1 + \tanh(w_{ji}x_i)}{1 - \tanh(w_{ji}x_i)}$$

Using the identity:

$$\frac{1 + \tanh(u)}{1 - \tanh(u)} = e^{2u}$$

we get:

$$\frac{z_j}{y_j} = e^{2\sum_{i=1}^n w_{ji}x_i}$$

so that:

$$\gamma_j := \frac{1}{2} \log \left(\frac{z_j}{y_j} \right) = \sum_{i=1}^n w_{ji}x_i$$

This gives:

$$\gamma = Wx$$

and if W is invertible:

$$x = W^{-1}\gamma$$

or:

$$x \approx W^\dagger \gamma$$

if W is not square and we use the pseudoinverse.

Log-Determinant of the Jacobian

The log-determinant of the Jacobian is required if this transformation is used in normalizing flows:

$$\log |\det J| = \sum_{j=1}^m \sum_{i=1}^n \log (1 - \tanh^2(w_{ji}x_i))$$

since:

$$\frac{d}{dx} \tanh(w_{ji}x_i) = w_{ji} (1 - \tanh^2(w_{ji}x_i))$$

Analysis of $y_j = \tanh\left(\sum_{i=1}^n W_{ji}x_i\right)$

Forward Transformation

This is a standard activation:

$$y_j = \tanh\left(\sum_{i=1}^n W_{ji}x_i\right)$$

or:

$$y = \tanh(Wx)$$

Inverse Transformation

The inverse exists coordinate-wise, provided $|y_j| < 1$:

$$\sum_{i=1}^n W_{ji}x_i = \tanh^{-1}(y_j)$$

So:

$$Wx = \tanh^{-1}(y)$$

If W is invertible:

$$x = W^{-1} \tanh^{-1}(y)$$

or:

$$x \approx W^\dagger \tanh^{-1}(y)$$

if W is not square.

Log-Determinant

The Jacobian is:

$$\frac{\partial y_j}{\partial x_i} = W_{ji} (1 - y_j^2)$$

Thus:

$$\log |\det J| = \log |\det W| + \sum_{j=1}^m \log(1 - y_j^2)$$

Plan: Unify the Ergodic and Canonical Space Hierarchies

Goal

Replace the current split between the canonical hierarchy (`Space`, `InputSpace`, `PerceptualSpace`, `ConceptualSpace`, `SymbolicSpace`, `SyntacticSpace`, `OutputSpace`) and the ergodic/plain-tensor hierarchy (`DerivedSpace`, `DerivedInputSpace`, `DerivedConceptualSpace`, `DerivedOutputSpace`) with one configurable space system.

The end state should keep current behavior for both:

- the object-encoded `BasicModel` pipeline
- the plain-tensor `DerivedModel` pipeline

without maintaining two separate implementations of shape handling, vector-set selection, parameter updates, and reverse-pass behavior.

Current Split To Remove

1. Tensor representation is duplicated. Canonical spaces depend on `TheObjectEncoding`, embedded dimensions, and codebook position padding. Derived spaces operate on raw tensors and flatten aggressively.
2. Vector-set behavior is duplicated. Canonical spaces use `VectorSet` with object-encoded embeddings. Derived spaces choose between `VectorSet` and `UnquantizedVSet` for raw tensors.
3. Layer orchestration is duplicated. `ConceptualSpace` and `DerivedConceptualSpace` both manage sigma/norm/ reverse-pass logic, but each does it with different shape conventions.
4. Model construction is split. `BasicModel.create(...)` builds the canonical chain, while `DerivedModel` rebuilds a smaller three-stage pipeline with separate assumptions.
5. Training hooks are inconsistent. Derived spaces expose `getParameters()` and `paramUpdate()` for ergodic layers; canonical spaces mostly rely on `nn.Module.parameters()` and do not expose the same contract.

Target Architecture

1. Introduce a shared `SpaceCore`

Create a single base class to replace both `Space` and `DerivedSpace`.

Responsibilities:

- own `inputShape`, `outputShape`, `nVectors`, `nDim`, `reversePass`
- provide shared `forwardBegin`, `forwardEnd`, `reverseBegin`, `reverseEnd`
- provide shared flatten/reshape helpers
- provide shared vector-set lifecycle and parameter-update hooks

This class should not hard-code `ObjectEncoding`. It should delegate tensor representation rules to a codec/adapter.

2. Split representation from behavior

Introduce a tensor codec abstraction, for example `SpaceCodec`, with two concrete modes:

- `ObjectEncodedCodec`
- `RawTensorCodec`

Responsibilities:

- compute embedded input/output dimensions

- add or strip object-encoding payloads
- validate shapes
- flatten and reshape consistently
- expose any symbol/object padding rules

This removes the biggest difference between `Space` and `DerivedSpace`.

3. Split vector lookup from space logic

Create a vector-backend abstraction, for example `VectorBackend`, to wrap:

- `VectorSet`
- `UnquantizedVSet`
- `ReversibleDictionary`

Responsibilities:

- `forward(...)`
- `reverse(...)`
- `create(...)`
- `addVectors(...)`
- `getParameters()`
- `paramUpdate()`

This lets the space hierarchy choose quantized vs pass-through vs dictionary behavior without forking the entire space class.

4. Keep specialized spaces as thin policies

Refactor the concrete spaces so they differ only in the transform they apply:

- `InputSpace`: lookup / dictionary / identity lifting
- `PerceptualSpace`: pi transform + optional attention
- `ConceptualSpace`: sigma transform + optional norm + optional attention
- `SymbolicSpace`: activation reduction / discretization / dereference
- `SyntacticSpace`: sequence-preserving symbolic pass-through
- `OutputSpace`: linear projection

The current `DerivedInputSpace`, `DerivedConceptualSpace`, and `DerivedOutputSpace` should become temporary wrappers or disappear entirely once the shared base can be configured with `RawTensorCodec`.

Migration Plan

Phase 1: Freeze behavior with tests

Add regression coverage before refactoring:

- canonical `InputSpace`, `ConceptualSpace`, `OutputSpace` forward shapes
- derived `DerivedInputSpace`, `DerivedConceptualSpace`, `DerivedOutputSpace` forward shapes
- reverse-path coverage for `reversePass=True`
- ergodic vs non-ergodic conceptual behavior
- quantized vs unquantized vector behavior
- symbol-processing cases that depend on `TheObjectEncoding.objectSize`

Acceptance gate: existing `BasicModel` and `DerivedModel` runs produce the same tensor shapes and compatible losses as before.

Phase 2: Extract shared shape/parameter plumbing

Refactor `Space` and `DerivedSpace` into one implementation without changing public behavior.

Tasks:

- add `SpaceCore`
- move shared begin/end/flatten/reshape logic there
- move shared vector-set creation there
- standardize `getParameters()` and `paramUpdate()` across all spaces
- keep current class names as wrappers while the internals change

Acceptance gate: all existing space classes still instantiate under the same constructor names.

Phase 3: Introduce codec layer

Move all `TheObjectEncoding`-specific shape math out of canonical spaces and into `ObjectEncodedCodec`.

Tasks:

- replace direct calls to `TheObjectEncoding.getEmbeddingSize(...)` in base space plumbing
- move positional/object padding logic behind codec methods
- make raw-tensor spaces use `RawTensorCodec`

Acceptance gate: `ConceptualSpace` and `DerivedConceptualSpace` differ only in transform and configuration, not in base tensor plumbing.

Phase 4: Unify conceptual space first

Start with `ConceptualSpace` and `DerivedConceptualSpace`, because they contain the densest overlap.

Tasks:

- define one conceptual implementation with configuration flags for: `ergodic`, `invertible`, `hasNorm`, `hasAttention`, `useVQ`, `processSymbols`, `reversePass`
- preserve current sigma-layer selection: `SigmaLayer`, paired reverse sigma, `ReversibleSigmaLayer`, or `LinearLayer` + `Tanh` fallback
- preserve current parameter exposure for ergodic layers

Acceptance gate: both current conceptual classes become wrappers around the same internals.

Phase 5: Unify input and output spaces

Tasks:

- merge `InputSpace` and `DerivedInputSpace` via configurable lookup backend
- merge `OutputSpace` and `DerivedOutputSpace` via shared linear projection plumbing
- keep dictionary-backed input and pass-through input as backend options, not new space classes

Acceptance gate: the three-stage derived pipeline can be built entirely from unified spaces.

Phase 6: Unify model assembly

Do not try to collapse `BasicModel` and `DerivedModel` into one monolith. Instead, extract a shared builder or pipeline factory.

Tasks:

- create a shared model-construction helper that instantiates spaces from a config object
- let `BasicModel` remain the richer topology
- let `DerivedModel` remain a smaller topology, but built from the same space implementations
- keep existing CLI/config flags working

Acceptance gate: `DerivedModel.create(...)` no longer references `Derived*Space` classes.

Phase 7: Remove the temporary fork

Tasks:

- delete `DerivedSpace`, `DerivedInputSpace`, `DerivedConceptualSpace`, `DerivedOutputSpace`
- remove compatibility wrappers once callers have migrated
- clean up comments that refer to a temporary coexistence plan

Acceptance gate: there is exactly one space hierarchy in `BasicModel.py`.

Design Constraints

1. Preserve state-dict compatibility where feasible. If module names must change, add a migration shim or a documented one-time conversion path.
2. Do not make `ObjectEncoding` mandatory for raw-tensor experiments. The derived path exists for a reason: it avoids object-padding overhead.
3. Keep ergodic parameter updates explicit. The `paramUpdate()` contract is part of the behavior for `SigmaLayer`-based training and cannot disappear into generic optimizer code.
4. Keep topology separate from tensor representation. The canonical pipeline has more stages than the derived pipeline; unifying spaces should not force a single model graph.
5. Refactor one layer family at a time. `ConceptualSpace` is the highest-value merge. `PerceptualSpace`, `SymbolicSpace`, and `SyntacticSpace` can follow after the core plumbing is stable.

Concrete Deliverables

- one shared base class for all spaces
- one codec abstraction for raw vs object-encoded tensors
- one vector backend abstraction for quantized vs pass-through vs dictionary
- unified conceptual/input/output implementations
- shared model builder used by both `BasicModel` and `DerivedModel`
- regression tests covering both pipelines
- deletion of the `Derived*` hierarchy

Suggested Execution Order

1. Add tests around current canonical and derived space behavior.
2. Introduce `SpaceCore` without changing public constructors.
3. Introduce codec abstraction and move `ObjectEncoding` shape logic into it.
4. Merge conceptual-space internals.
5. Merge input/output-space internals.
6. Extract shared model builder.
7. Delete `Derived*` classes and dead code.

Definition of Done

This work is complete when:

- `BasicModel` and `DerivedModel` both run on the unified space implementation
- there is no duplicate shape/vector-set plumbing between canonical and ergodic paths
- the temporary “Kept separate ... until a future unification pass” comment can be removed
- existing training modes still work: ergodic, certainty-weighted, quantized/unquantized, reverse-pass, invertible, and symbol-processing modes

Appendix: Tensor Representation Analysis

The two representations in the current code

There are currently two different meanings attached to a “vector” in the space hierarchy.

1. Canonical / object-encoded representation

A vector is not just “what” the object is. It is:

- semantic coordinates (**what**)
- positional encoding (**where**)
- temporal encoding (**when**)

In code, this is implemented by `ObjectEncoding`, where:

- `objectSize = nWhere + nWhen`
- `embeddingSize = nDim + objectSize`

So a canonical tensor has the form:

- `X_canonical` in $\mathbb{R}[B, N, D + S]$

where:

- `B` = batch size
- `N` = number of vectors in the space
- `D` = semantic dimension for that space
- `S` = `TheObjectEncoding.objectSize`

The last `S` channels are reserved for metadata. Several canonical spaces explicitly strip and reattach that suffix when computing norms, symbols, or reverse transforms.

2. Ergodic / raw-tensor representation

A vector is only the semantic state:

- `X_raw` in $\mathbb{R}[B, N, D]$

There is no in-band **where** or **when** payload. If position or time matters, it must be recovered from index convention, batch order, or external logic.

The derived hierarchy then frequently flattens this to:

- `X_flat` in $\mathbb{R}[B, N * D]$

applies a dense transform, and reshapes back to $[B, N', D']$.

Theoretical difference

The canonical path represents an object as a tuple:

- `object = (what, where, when)`

The raw path represents an object as:

- `object = (what)`

This is not just an implementation detail. It changes what the tensor means.

Canonical tensors carry explicit object identity context inside the channel dimension. That makes the representation closer to a typed bundle: the tensor does not only say “this feature value exists”, it also says “this feature value belongs to object *i* at position *p* and time *t*”.

Raw tensors are a projection of that richer object into its semantic component. They are closer to a plain state vector. That is simpler, but it also means the representation is only fully interpretable if index position and timing are known elsewhere.

Architectural difference in this codebase

The canonical hierarchy is architected around object-preserving transforms. Most spaces operate on rank-3 tensors and preserve the object axis explicitly:

- `InputSpace`: `[B, input_count, input_dim] -> [B, nInput, input_embedding]`
- `PerceptualSpace`: `[B, N, embedding] -> [B, N', embedding]`
- `ConceptualSpace`: `[B, N, embedding] -> [B, N', embedding]`
- `SymbolicSpace`: often keeps the object suffix while collapsing semantic magnitude into a symbol channel

Even when canonical spaces compute on semantic channels only, they often preserve or reconstruct the trailing object metadata.

The ergodic hierarchy is architected around plain tensors and whole-space projections:

- `DerivedInputSpace`: `[B, N, D] -> [B, N, D]`
- `DerivedConceptualSpace`: `[B, N, D] -> flatten -> [B, N * D] -> transform -> reshape`
- `DerivedOutputSpace`: same flatten/project/reshape pattern

So the major architectural split is:

- canonical path: object-aware tensors, metadata in-band, object axis preserved
- derived path: semantic-only tensors, metadata absent, object axis often folded into the feature axis for dense transforms

What PyTorch is doing underneath

PyTorch tensors are strided views over storage. The important practical consequence for this refactor is that shape changes are not all equal.

Relevant PyTorch behavior:

- `unsqueeze()` is a view op. It inserts a size-1 dimension without copying.
- basic slicing is usually a view op. For example, taking `x[:, :, :-S]` generally reinterprets the same storage.
- `view()` requires compatible strides and shared storage layout.
- `reshape()` returns a view when possible, but may allocate a copy when the strides are incompatible.
- `contiguous()` returns the original tensor if it is already contiguous; otherwise it allocates a copied contiguous tensor.
- `torch.cat()` allocates a new tensor for the concatenated result.
- `nn.Linear` operates on the last dimension and preserves all leading dimensions.

That means the current code has two distinct cost profiles:

1. Cheap metadata-only shape edits

These include:

- `unsqueeze`
- many slices
- some reshapes when the tensor is already contiguous

These are mostly stride rewrites, not real data movement.

2. Real allocation and copy points

These include:

- `torch.cat(...)`
- `F.pad(...)`
- `reshape(...)` when the input is not compatible for a view
- `contiguous()` on non-contiguous inputs

In this codebase, the canonical path pays this cost whenever it pads or concatenates object metadata, and the derived path may pay it when flattening or reshaping non-contiguous tensors.

Efficiency tradeoffs

Canonical / object-encoded strategy Strengths:

- preserves object identity, position, and time in-band
- keeps object axis explicit, which is conceptually aligned with spaces, codebooks, dereferencing, and symbolic processing
- makes it easier to express operations that must distinguish semantic content from object metadata

Costs:

- memory overhead of $S = \text{objectSize}$ extra channels on every vector
- extra memory traffic from repeated strip/preserve/reattach patterns
- extra allocations from `cat` / `pad`
- more autograd bookkeeping because those tensor constructions become part of the graph

The overhead factor is approximately:

- $(D + S) / D$

So if D is small, the metadata tax is large. If D is large, the tax becomes less important.

Raw / ergodic strategy Strengths:

- no per-vector metadata overhead
- fewer padding and concatenation operations
- simpler memory layout
- usually better for throughput in experiments where only semantic values matter

Costs:

- position and time are no longer explicit tensor fields
- flattening $[B, N, D]$ into $[B, N * D]$ mixes object structure into one dense axis
- a whole-space dense transform can be more expensive than a per-object transform if N is large
- reverse mapping is less semantically explicit because the tensor alone no longer says which channels were metadata and which were content

So the raw strategy is cheaper in channel count, but the architectural choice to flatten whole spaces can create larger dense matrices and can erase useful structure.

Mapping between the two representations

Let:

- $S = \text{TheObjectEncoding.objectSize}$
- $X_{\text{raw}} \in \mathbb{R}[B, N, D]$
- $X_{\text{can}} \in \mathbb{R}[B, N, D + S]$

Then the forward map from raw to canonical is:

- $X_{\text{can}} = \text{Encode}(X_{\text{raw}})$
- in the current code this is approximately: `TheObjectEncoding.forward(X_raw, pad=True)`

Conceptually:

- $\text{Encode}(x_i) = \text{concat}(x_i, \text{where}(i), \text{when}(i \text{ or } t))$

This map is injective if **where** and **when** are available or can be regenerated from context.

The reverse map from canonical to raw is:

- `X_raw = Strip(X_can)`
- in the current code this is approximately: `X_can[..., :-S]` or `TheObjectEncoding.slice(...)` per vector

This map is lossy, because **where** and **when** are discarded.

So the relationship is:

- canonical -> raw: projection
- raw -> canonical: embedding, but only if metadata can be supplied

Architectural mapping between operations

If a canonical operator only uses semantic channels, then it has a direct raw equivalent:

1. strip metadata
2. apply the raw operator
3. optionally reattach metadata

For example:

- canonical conceptual path: `x[..., :-S] -> sigma/pi/linear -> maybe VQ -> concat metadata`
- raw conceptual path: `x -> sigma/pi/linear -> maybe VQ`

If a canonical operator actually uses **where** or **when**, then the raw equivalent must either:

- receive those values as a side-channel, or
- reconstruct them from index/time convention

This is why a clean unification target should not simply pick one of the two representations. It should model them explicitly.

Best unification target

The cleanest shared abstraction is not “always concatenate metadata” and not “always drop metadata”. It is:

- semantic tensor: `[B, N, D]`
- optional object metadata tensor: `[B, N, S]`

with a codec deciding whether the operational representation is:

- fused: `[B, N, D + S]`
- split: `([B, N, D], [B, N, S])`
- semantic-only: `[B, N, D]`

That would let the unified hierarchy preserve the current canonical behavior without forcing the ergodic path to pay the metadata tax.

Practical implication for the refactor

The duplication is not only about shape helpers. It exists because the codebase currently mixes two distinct choices:

1. how object metadata is represented
2. how a space transforms tensors

Those should be separated.

The refactor should therefore:

- move representation into a codec layer
- keep transforms (**pi**, **sigma**, **linear**, symbolic reduction) independent of that codec

- make metadata explicit enough that canonical behavior remains possible
- avoid forcing raw experiments through repeated `cat` / `slice` / `pad` operations

PyTorch reference points

- Tensor views: https://docs.pytorch.org/docs/main/tensor_view.html
- `torch.Tensor.view`: <https://docs.pytorch.org/docs/stable/generated/torch.Tensor.view.html>
- `torch.reshape`: <https://docs.pytorch.org/docs/stable/generated/torch.reshape.html>
- `torch.unsqueeze`: <https://docs.pytorch.org/docs/stable/generated/torch.unsqueeze.html>
- `torch.cat`: <https://docs.pytorch.org/docs/2.9/generated/torch.cat.html>
- `nn.Linear`: <https://docs.pytorch.org/docs/stable/generated/torch.nn.modules.linear.Linear.html>
- tensor storage model: <https://docs.pytorch.org/docs/stable/tensors.html>